

10.7 Strings

Because of the "equivalence" of arrays and pointers, it is extremely common to refer to and manipulate strings as character pointers, or `char *`'s. It is so common, in fact, that it is easy to forget that strings are arrays, and to imagine that they're represented by pointers. (Actually, in the case of strings, it may not even matter that much if the distinction gets a little blurred; there's certainly nothing wrong with referring to a character pointer, suitably initialized, as a "string.") Let's look at a few of the implications:

1. Any function that manipulates a string will actually accept it as a `char *` argument. The caller may pass an array containing a string, but the function will receive a pointer to the array's (string's) first element (character).
2. The `%s` format in `printf` expects a character pointer.
3. Although you have to use `strcpy` to copy a string from one array to another, you can use simple pointer assignment to assign a string to a pointer. The string being assigned might either be in an array or pointed to by another pointer. In other words, given

```
char string[] = "Hello, world!";
char *p1, *p2;
```

both

```
p1 = string
```

and

```
p2 = p1
```

are legal. (Remember, though, that when you assign a pointer, you're making a copy of the pointer but *not* of the data it points to. In the first example, `p1` ends up pointing to the string in `string`. In the second example, `p2` ends up pointing to the same string as `p1`. In any case, after a pointer assignment, if you ever change the string (or other data) pointed to, the change is "visible" to *both* pointers.

4. Many programs manipulate strings exclusively using character pointers, never explicitly declaring any actual arrays. As long as these programs are careful to allocate appropriate memory for the strings, they're perfectly valid and correct.

When you start working heavily with strings, however, you have to be aware of one subtle fact.

When you initialize a character array with a string constant:

```
char string[] = "Hello, world!";
```

you end up with an array containing the string, and you can modify the array's contents to your heart's content:

```
string[0] = 'J';
```

However, it's possible to use string constants (the formal term is *string literals*) at other places in your code. Since they're arrays, the compiler generates pointers to their first elements when they're used in expressions, as usual. That is, if you say

```
char *p1 = "Hello";
int len = strlen("world");
```

it's almost as if you'd said

```
char internal_string_1[] = "Hello";
char internal_string_2[] = "world";
char *p1 = &internal_string_1[0];
int len = strlen(&internal_string_2[0]);
```

Here, the arrays named `internal_string_1` and `internal_string_2` are supposed to suggest the fact that the compiler is actually generating little temporary arrays every time you use a string constant in your code. *However*, the subtle fact is that the arrays which are ``behind" the string constants are *not* necessarily modifiable. In particular, the compiler may store them in read-only-memory. Therefore, if you write

```
char *p3 = "Hello, world!";
p3[0] = 'J';
```

your program may crash, because it may try to store a value (in this case, the character 'J') into nonwritable memory.

The moral is that whenever you're building or modifying strings, you have to make sure that the memory you're building or modifying them in is writable. That memory should either be an array you've allocated, or some memory which you've dynamically allocated by the techniques which we'll see in the next chapter. Make sure that no part of your program will ever try to modify a string which is actually one of the unnamed, unwritable arrays which the compiler generated for you in response to one of your string constants. (The only exception is array initialization, because if you write to such an array, you're writing to the array, not to the string literal which you used to initialize the array.)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail](#) [feedback](#)